

P13

ReAct: sinergia entre razonar y actuar en modelos de lenguaje

ReAct: Synergizing Reasoning and Acting in Language Models

Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, Yuan Cao · 2022 ·
arXiv:2210.03629 · ICLR 2023

Ficha pedagógica del Artificial Intelligence Evolution Program. No es el paper original: es una guía de lectura en español que enlaza a la fuente primaria. Consultado 2026-08-16.

P13 — ReAct

El modelo deja de ser un generador de texto y pasa a ser el **controlador** de un bucle que observa el mundo y actúa sobre él. Es el paper donde empieza, en sentido técnico, el agente.

Nivel: L2 · Motor: `react` · Notebook: `P13_react.ipynb`

1. Identificación

Campo	Valor
Título original	<i>ReAct: Synergizing Reasoning and Acting in Language Models</i>
Autoría	Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, Yuan Cao
Año	2022 (arXiv) · 2023 (ICLR)
Venue	arXiv:2210.03629 · ICLR 2023
Fuente primaria	arXiv:2210.03629
Acceso	Abierto
Fecha de consulta	2026-08-16

2. Problema anterior

Existían dos líneas de trabajo que no se hablaban:

- **Razonar sin actuar.** El razonamiento en cadena (Wei et al., 2022) mejora tareas de varios pasos haciendo que el modelo escriba su razonamiento. Pero ese razonamiento ocurre **dentro** del modelo, sin contacto con el mundo: si un hecho intermedio es falso, nada lo corrige y el error se propaga con toda la apariencia de rigor.
- **Actuar sin razonar.** Los modelos que emiten acciones sobre un entorno (navegar, buscar, manipular) no descomponen la tarea ni replantean el plan cuando una acción no da lo esperado.

Faltaba el acoplamiento: que **lo que se observa condicione lo que se piensa a continuación**.

3. Propuesta

Intercalar dos tipos de salida en la misma secuencia generada:

Thought → Action → Observation → Thought → Action → Observation → ... → Finish

- **Thought** — texto libre: descomponer, planificar, replantear, decidir cuándo parar.
- **Action** — llamada a una herramienta del entorno (buscar, consultar, moverse).
- **Observation** — resultado real devuelto por el entorno; **no lo genera el modelo**.

Lo notable es la sinergia bidireccional: el razonamiento decide qué acción tomar, y la observación real ancla el siguiente razonamiento. El método se aplica mediante prompting con pocos ejemplos, sin entrenamiento adicional.

4. Intuición sin fórmulas

Un investigador que piensa en voz alta mientras consulta archivos. No decide toda la investigación de antemano ni abre carpetas al azar: piensa qué necesita, lo busca, lee lo que encontró y ese hallazgo cambia lo que busca después.

Dónde deja de funcionar la analogía: el investigador sabe cuándo su fuente es poco fiable. El bucle no: si la herramienta devuelve un dato erróneo, lo propaga sin dudar. Lo comprueba el notebook.

5. Matemática mínima

No hay ecuación nueva. Lo que cambia es el **espacio de acciones** del modelo:

Generación estándar:

$a_t \in \text{vocabulario de tokens}$

ReAct:

$a_t \in \text{vocabulario} \cup \text{espacio de acciones del entorno}$

$\text{contexto}_t = (\text{objetivo}, o_1, a_1, \dots, o_{\{t-1\}}, a_{\{t-1\}})$

$a_t \sim p_\theta(\cdot \mid \text{contexto}_t)$

Las observaciones **o** entran en el contexto pero **no** las genera el modelo: vienen del entorno. Ese es el anclaje.

6. Arquitectura o flujo

pregunta: «¿cuántos habitantes tiene la capital de Francia?»

Thought: no sé qué ciudad es; primero busco la capital	
Action: buscar("capital de Francia")	
Observation: "París"	← DEL ENTORNO
Thought: es París; ahora busco su población	
Action: buscar("población de París")	
Observation: "2 100 000"	← DEL ENTORNO
Thought: tengo el dato, puedo responder	
Action: finish("2 100 000")	

Sin el primer paso, la pregunta es irresoluble con una sola búsqueda.

7. Qué observar en el paper original

- La **comparación de cuatro modos**: solo razonar (CoT), solo actuar (Act), ReAct, y las combinaciones ReAct+CoT-SC. La tabla que las compara es el resultado central.
- Las **trazas de ejemplo** en los apéndices: contienen tanto casos de éxito como fallos, y los fallos son más instructivos.
- El análisis de **modos de error**: alucinación de razonamiento, bucles repetitivos, búsquedas poco informativas.
- Los **cuatro entornos**: HotpotQA y FEVER (razonamiento sobre conocimiento), ALFWorld y WebShop (toma de decisiones interactiva). La variedad importa: demuestra que el patrón no es específico de la búsqueda de texto.

8. Evidencia y resultados

- En **HotpotQA** y **FEVER**, ReAct reduce la alucinación frente a CoT puro, porque las observaciones de la Wikipedia anclan los hechos; en exactitud pura, CoT puede ganar en algunos casos, y el artículo lo reconoce en lugar de ocultarlo.
- En **ALFWorld** y **WebShop** —tareas interactivas— ReAct supera de forma clara a los métodos de solo actuar (imitación y refuerzo), con muy pocos ejemplos en el prompt.
- La combinación de ReAct con autoconsistencia obtiene los mejores resultados globales.

Las cifras por entorno y método están en las tablas del artículo. Verificarlas allí: la comparación es matizada y resumirla como «ReAct gana siempre» es incorrecto.

La miniatura de este eje aporta la evidencia estructural: una pregunta de dos saltos es irresoluble con una sola búsqueda, y la traza muestra cómo la primera observación **construye** la segunda consulta.

9. Impacto

- Es el patrón de referencia de casi todos los frameworks de agentes que vinieron después.
- Introduce la **traza auditable** como artefacto de primera clase: se puede revisar qué hizo el sistema y por qué, sin abrir los pesos.
- Convierte «llamar a una herramienta» en parte del bucle de razonamiento, no en un postprocesado.
- Prepara el terreno para Toolformer (aprender **cuándo** llamar) y para los sistemas agentic (memoria, reflexión, presupuesto).

10. Limitaciones

1. **Sin criterio de parada, el bucle no termina.** Es el fallo operativo número uno: la herramienta falla, el agente reintenta, el coste crece sin límite.
2. **La calidad está acotada por la de las herramientas.** Si la fuente miente, el agente miente con seguridad y con traza.
3. **La traza no es fiel.** El texto del «pensamiento» es una generación más y puede no describir el proceso real del modelo. Sirve para depurar, no como prueba.
4. **Coste:** cada paso es una llamada al modelo más una a la herramienta. La latencia se multiplica.

5. **Superficie de ataque:** una observación puede contener texto que intente redirigir al agente (inyección de prompt indirecta).
6. **Depende de un modelo grande:** con modelos pequeños, la calidad de los pensamientos se degrada rápidamente.
7. **Sin memoria entre episodios:** cada tarea empieza de cero.

11. Errores comunes

Error	Corrección
«La traza explica cómo razonó el modelo»	Es texto generado. Útil para auditar decisiones, no una prueba del proceso interno.
«ReAct siempre supera a CoT»	Depende de la tarea. El propio paper reporta casos donde CoT es mejor.
«ReAct es un framework»	Es un patrón de prompting . Los frameworks lo implementan; no son el paper.
«Un agente sin límite de pasos es más capaz»	Es más caro y más frágil. El criterio de parada es una función de seguridad, no una limitación.
«Si el agente cita una observación, el dato es correcto»	El dato es tan fiable como la herramienta que lo produjo.
«ReAct requiere entrenamiento especial»	Se aplica con pocos ejemplos en el prompt, sin entrenar.

12. Relación con trabajos anteriores

- **P10 GPT-3 (2020)** — el aprendizaje en contexto que hace viable aplicar el patrón con pocos ejemplos.
- **Wei et al. (2022), Chain-of-Thought** — razonar explícitamente en el prompt. [arXiv:2201.11903](#)
- **Wang et al. (2022), autoconsistencia** — muestrear varios razonamientos y votar. [arXiv:2203.11171](#)
- **P11 RAG (2020)** — recuperar como paso fijo; aquí pasa a ser una acción decidida por el modelo.

13. Relación con trabajos posteriores

- **P14 Toolformer (2023)** — aprender cuándo llamar a la herramienta.
- **Reflexion (Shinn et al., 2023)** — añadir autocritica y memoria entre intentos. [arXiv:2303.11366](#)
- **Generative Agents (Park et al., 2023)** — memoria episódica y recuperación por relevancia. [arXiv:2304.03442](#)
- **P16 Sistemas agentic** — el bucle convertido en sistema.
- **Model Context Protocol** — estandarización posterior del acceso a herramientas. [modelcontextprotocol.io](#)

14. Notebook asociado

`P13_react.ipynb`

Qué implementa: la comparación entre una estrategia de solo actuar y el bucle completo sobre una pregunta de dos saltos; un bucle sin criterio de parada como anti-patrón; una versión con límite de pasos,

detección de repetición y escalamiento; y un experimento donde la base de conocimiento devuelve un dato corrupto.

Qué NO implementa: ningún modelo de lenguaje. Los «pensamientos» están escritos a mano. En el paper los genera el modelo, y esa es precisamente la parte que puede fallar.

```
ai-evolution paper-lab P13 --seed 7
```

15. Actividades Bloom

Nivel	Actividad
Recordar	Escribe los tres elementos del bucle y di cuál no genera el modelo.
Explicar	Explica por qué una pregunta de dos saltos falla con una sola búsqueda.
Aplicar	Añade una tercera consulta encadenada al notebook y comprueba que la traza sigue siendo legible.
Analizar	Corrompe la base de conocimiento y analiza cómo se propaga el error por la traza.
Evaluar	Un agente resuelve una tarea en 20 pasos y otro en 4. ¿Cuál es mejor? Di qué necesitas saber antes de responder.
Crear	Diseña un verificador que contraste cada observación con una segunda fuente, y estima su coste en llamadas.

16. Autoevaluación

1. ¿Qué aporta la observación que no puede aportar el razonamiento interno?
2. ¿Por qué el razonamiento en cadena puro alucina hechos?
3. ¿Qué tres mecanismos evitan que un bucle se vuelva infinito?
4. ¿Por qué una traza legible no es una explicación del modelo?
5. ¿Qué riesgo de seguridad introduce leer observaciones de fuentes externas?
6. ¿En qué se diferencia ReAct de RAG?
7. ¿Qué límite de ReAct ataca directamente el paper siguiente?

17. Respuestas esperadas

1. Información del mundo que el modelo no tiene o tiene desactualizada, y una señal de corrección: si la acción no da lo esperado, el siguiente pensamiento puede replantear.
2. Porque genera los hechos intermedios desde sus parámetros, sin contrastarlos. Un eslabón falso se integra en la cadena y arrastra al resto con apariencia de solidez.
3. Límite de pasos, detección de estados o consultas repetidas, y escalamiento a un humano ante fallo persistente de la herramienta. Se aceptan también: presupuesto de tokens y de coste.
4. Porque es texto generado por el mismo proceso que produce la respuesta. Puede ser una racionalización posterior y no la causa de la acción.
5. Inyección de prompt indirecta: la observación puede contener instrucciones dirigidas al agente. Todo contenido observado debe tratarse como dato, nunca como instrucción.
6. En RAG la recuperación es un paso fijo del pipeline. En ReAct es una **acción** que el modelo decide ejecutar, cuántas veces y con qué consulta.

7. Que las herramientas y cuándo usarlas se especifican a mano en el prompt. Toolformer aprende ese «cuándo» de forma autosupervisada.

18. Fuentes primarias

- Yao, S. et al. (2023). *ReAct: Synergizing Reasoning and Acting in Language Models*. **ICLR 2023**. arXiv:2210.03629 · consultado 2026-08-16.
- Wei, J. et al. (2022). *Chain-of-Thought Prompting Elicits Reasoning in Large Language Models*. **NeurIPS 2022**. arXiv:2201.11903 · consultado 2026-08-16.
- Shinn, N. et al. (2023). *Reflexion: Language Agents with Verbal Reinforcement Learning*. arXiv:2303.11366 · consultado 2026-08-16.



Evaluación — P13 · ReAct: sinergia entre razonar y actuar en modelos de lenguaje

Generado por `python scripts/generate_papers.py`. Se evalúa comprensión histórica, lectura crítica e interpretación — no memorización de definiciones.

Paper: *ReAct: Synergizing Reasoning and Acting in Language Models* (2022, arXiv:2210.03629 · ICLR 2023) · **Nivel:** L2

Parte A — Contexto histórico (20 pts)

- (10) Describe el estado del arte **inmediatamente anterior** a este paper y por qué era insuficiente. No menciones la solución del paper en tu respuesta.
- (10) Nombra un trabajo anterior del que este paper depende y explica qué le tomó prestado.

Parte B — Lectura crítica (20 pts)

- (10) Localiza en el paper original una afirmación **cuantitativa** y reescríbela indicando tarea, dataset, métrica, línea base y condiciones. Cita la tabla o sección.
- (10) Identifica una idea que hoy se asocia a este paper pero que **apareció después**. Aporta la referencia posterior con año.

Parte C — Interpretación matemática (15 pts)

- (15) Explica la ecuación central con tus palabras y señala qué ocurre en un caso límite (valor 0, dimensión muy grande, secuencia muy larga... según corresponda).

Parte D — Implementación e interpretación (25 pts)

- (10) Ejecuta `P13_react.ipynb` con **tres semillas** y reporta qué varía y qué se mantiene.
- (10) Reproduce el anti-patrón de la sección 11 y explica por qué produce una conclusión errónea.
- (5) Aporta la corrección con su evidencia.

Parte E — Límites y transferencia (20 pts)

- (10) Escribe una limitación de la **miniatura** y una del **paper original**. No pueden ser la misma idea.
- (10) Conecta este hito con el siguiente de la ruta: ¿qué quedó sin resolver que motivó el paso siguiente?

Rúbrica

Nivel	Descripción
A — Excelente	Distingue hecho documentado, simplificación didáctica e inferencia propia. Cita fuentes primarias con sección o tabla. Sus límites son propios, no copiados.
B — Suficiente	Explica el mecanismo y ejecuta la miniatura correctamente, pero repite los límites de la ficha y cita de forma imprecisa.
C — Insuficiente	Describe el paper con narrativa retrospectiva, atribuye ideas posteriores, o presenta la salida de la miniatura como reproducción del experimento original.

Criterio automático de rechazo

Se devuelve sin nota cualquier entrega que:

- atribuya al paper resultados, métricas o autores que no aparecen en la fuente primaria;
- presente la ejecución del notebook como reproducción de los resultados del paper;
- cite un paper que no se abrió (se comprueba pidiendo el número de figura o tabla).